ShopEZ

MERN Stack E-Commerce Application

PROJECT DOCUMENTATION

Student	Prasad Gadale
Degree	B.Tech - Computer Science & Engineering
Institution	Dr. D.Y. Patil College of Engineering & Technology, Kolhapur
Project Type	Full Stack Web Application
Technology	MongoDB, Express.js, React.js, Node.js
Payment Integration	Razorpay

This document describes the design, implementation, features, database structure, REST APIs, security approach, testing, setup procedure, and future scope of the ShopEZ e-commerce application.

Academic / Portfolio Project

Table of Contents

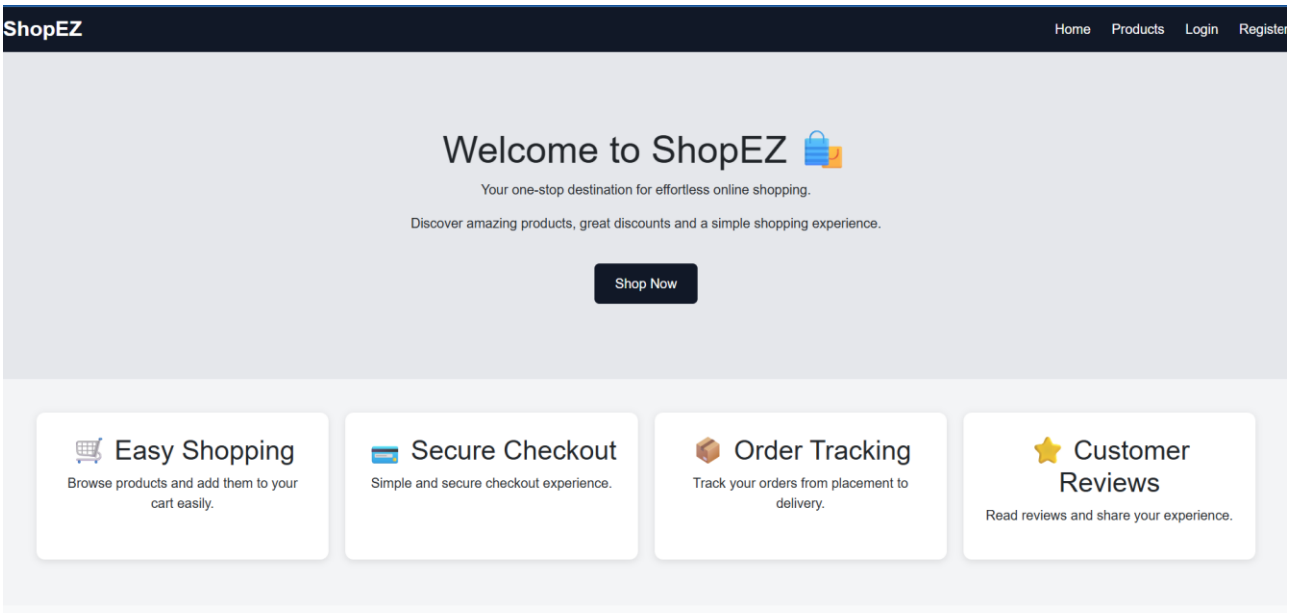
No.	Section
1	Introduction
2	Project Objectives
3	Problem Statement & Proposed Solution
4	Technology Stack
5	System Features
6	System Architecture
7	Project Structure
8	Database Design
9	User Module
10	Admin Module
11	Shopping Cart & Order Flow
12	Payment Integration
13	REST API Documentation
14	Authentication & Security
15	Validation & Error Handling
16	Testing
17	Installation & Setup
18	Environment Variables
19	Deployment & GitHub
20	Future Enhancements
21	Conclusion
22	References / Tools

1. Introduction

ShopEZ is a full-stack e-commerce web application developed using the MERN stack. The system provides a complete online shopping workflow in which users can register, log in, browse products, view product details, add products to a cart, update quantities, remove products, place orders, make online payments through Razorpay, and track or cancel eligible orders.

The application also contains an administrative module. Administrators can manage products and users, view all orders, update order statuses, and monitor business statistics such as total users, products, orders, pending orders, delivered orders, cancelled orders, and total sales.

The project follows a client-server architecture. The React frontend communicates with a Node.js/Express REST API, while MongoDB stores application data. JWT-based authentication protects user-specific and administrator-specific operations.



2. Project Objectives

- Build a practical and responsive e-commerce application using the MERN stack.
- Implement secure user registration and login with protected routes.
- Provide product browsing, product details, cart management, and checkout.
- Support Cash on Delivery and online payment through Razorpay.
- Maintain order history and allow cancellation before shipping.
- Provide administrator features for product, user, and order management.
- Implement stock validation and update inventory after successful orders.
- Create a clear REST API structure for communication between frontend and backend.
- Demonstrate a project suitable for academic submission, portfolio presentation, and interviews.

3. Problem Statement & Proposed Solution

Problem Statement: Traditional small-scale shopping applications often require separate systems for product management, customer accounts, carts, orders, inventory, and payments. A student-level project that combines these functions into one coherent application is useful for understanding real-world full-stack development.

Proposed Solution: ShopEZ provides a centralized web platform where customers can browse products and complete purchases, while administrators can manage the business side of the application. The backend exposes REST endpoints for authentication, products, cart, orders, reviews, users, admin operations, and payments.

The application keeps the cart associated with the authenticated user, validates product availability before checkout, calculates order totals on the server, records shipping information, reduces stock after order creation, and clears the cart after a successful order.

4. Technology Stack

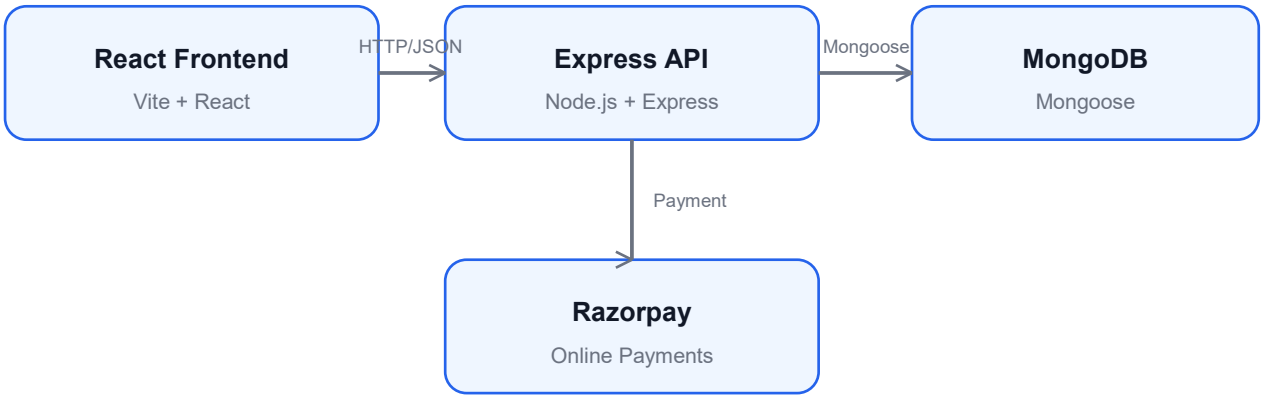
Layer	Technology	Purpose
Frontend	React.js	Component-based user interface
Build Tool	Vite	Fast frontend development and build
Styling	CSS	Page and component styling
Backend	Node.js	Server-side JavaScript runtime
API Framework	Express.js	REST API and routing
Database	MongoDB	Document-oriented data storage
ODM	Mongoose	MongoDB schemas, models and queries
Authentication	JWT	Authenticated API access
Payments	Razorpay	Online payment order and signature verification
HTTP Client	Axios	Frontend-to-backend API communication
Version Control	Git / GitHub	Source code management and portfolio hosting

5. System Features

- **User Authentication:** Register and log in using protected authentication.
- **Product Catalog:** View products, prices, categories, images, stock and product details.
- **Cart:** Add products, change quantities, remove products and clear the cart.
- **Checkout:** Capture name, phone, address, city, state and pincode.
- **Payments:** Cash on Delivery or Razorpay online payment.
- **Orders:** View order history, individual order details, payment status and shipping address. • **Cancellation:** Customers can cancel orders while the status is PLACED or CONFIRMED.
- **Reviews:** Product review functionality is included in the backend model/routes.
- **Admin Dashboard:** Statistics, product management, user management and order management.
- **Inventory:** Product stock is checked during cart/checkout operations and reduced after successful order placement.

6. System Architecture

ShopEZ uses a client-server architecture. The frontend is responsible for the user interface and interactions. The Express backend exposes REST endpoints and contains controllers, middleware and models. MongoDB stores persistent application data.



Request flow: React page → Axios API request → Express route → authentication/authorization middleware → controller → Mongoose model → MongoDB → JSON response → React UI.

The Razorpay payment flow is connected to the backend so that the server creates the payment order and verifies the payment signature before a paid ShopEZ order is created.

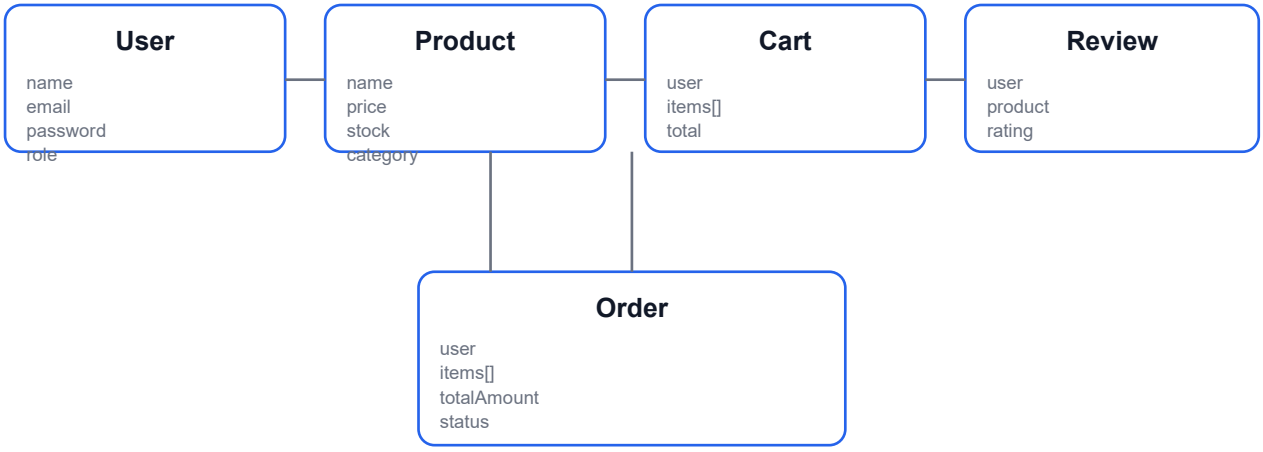
7. Project Structure

```
ShopEZ/
■■■ client/
■ ■■■ public/
■ ■■■ src/
■ ■ ■■■ components/
■ ■ ■■■ pages/
■ ■ ■■■ services/
■ ■ ■■■ App.jsx
■ ■ ■■■ main.jsx
■ ■■■ package.json
■ ■■■ vite.config.js
■■■ server/
■ ■■■ config/
■ ■■■ controllers/
■ ■■■ middleware/
■ ■■■ models/
■ ■■■ routes/
■ ■■■ package.json
■ ■■■ server.js ■■■ .gitignore
```

The frontend contains pages such as Home, Products, ProductDetails, Cart, Checkout, Orders, OrderDetails, Login, Register and administrator pages. The backend separates routes, controllers, models and middleware to keep the code organized and maintainable.

8. Database Design

MongoDB is used as the primary database. Mongoose schemas define the structure of application documents and relationships through ObjectId references.



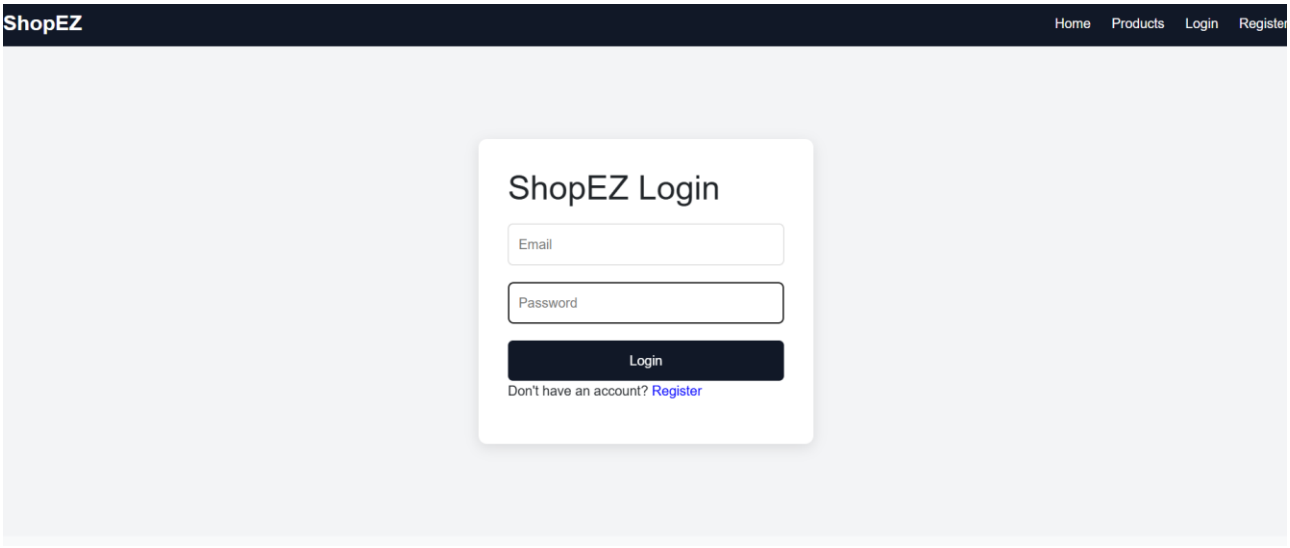
Collection / Model	Important Fields	Purpose
User	name, email, password, role	Stores customer and admin accounts
Product	name, description, price, discount, category, image, stock, seller, ratings	Stores product catalog information
Cart	user, items, total	Stores each user's current shopping cart
Order	user, items, totalAmount, shippingAddress, paymentMethod, paymentStatus, orderStatus	Stores placed orders
Review	user, product, rating, comment	Stores customer product reviews

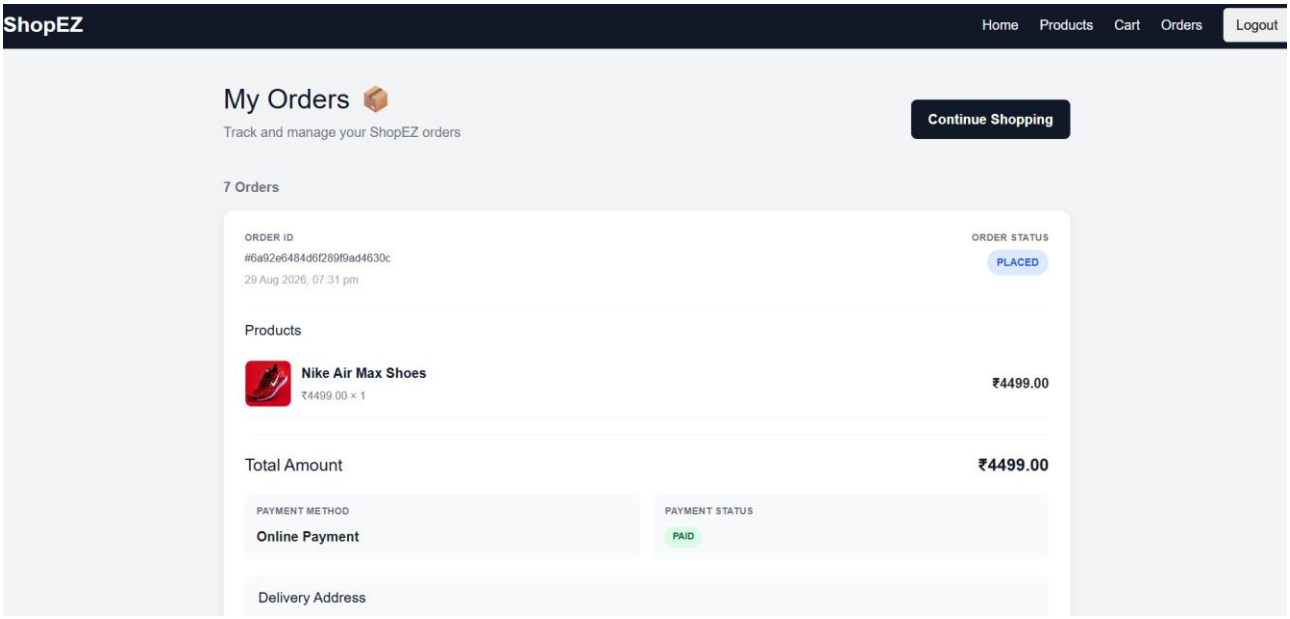
Order items store product reference, product name, quantity and price. Keeping the name and price inside an order item preserves the order information even if catalog information changes later.

9. User Module

- Register a new account and log in.
- Access protected pages only after authentication.
- Browse and filter available products.
- View detailed product information.
- Add products to the cart and manage quantities.
- Complete checkout with shipping information.
- Choose COD or online payment.
- View personal order history.
- Open an individual order and review products, amount, payment and delivery information.

• Cancel orders only when the order status is PLACED or CONFIRMED.

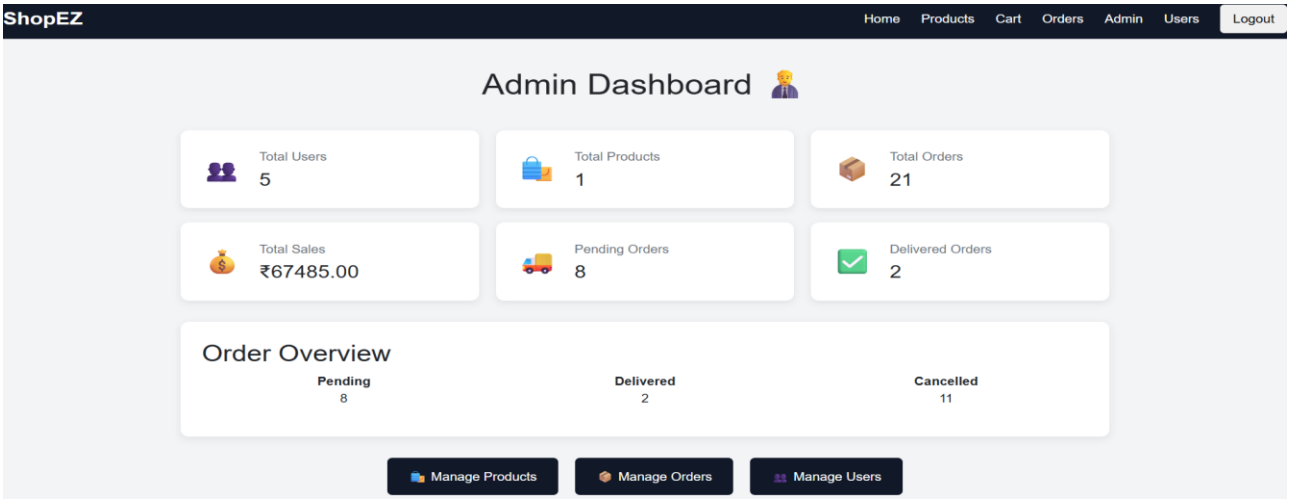


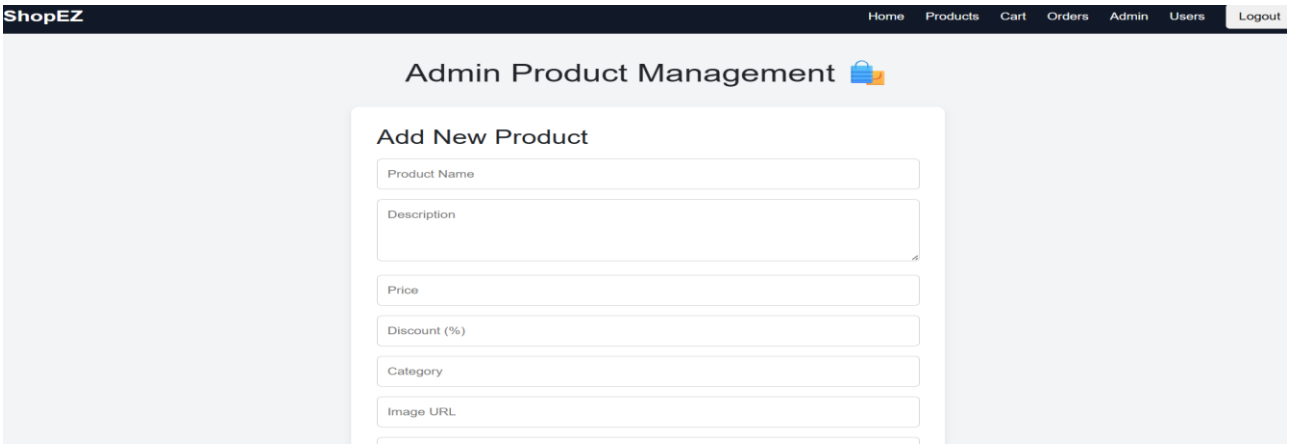


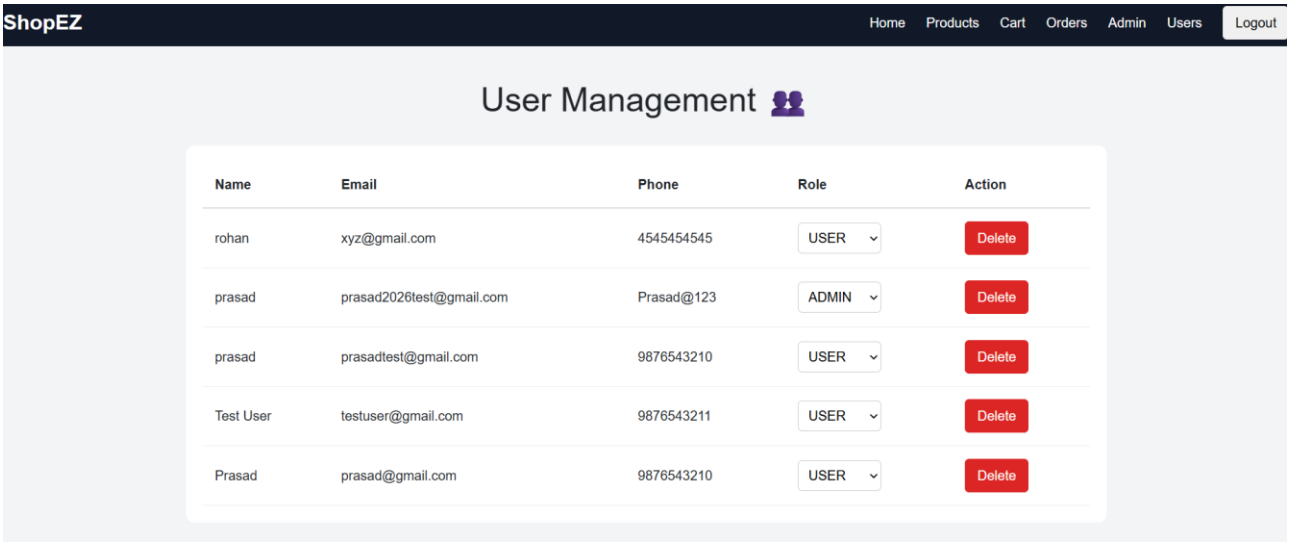
10. Admin Module

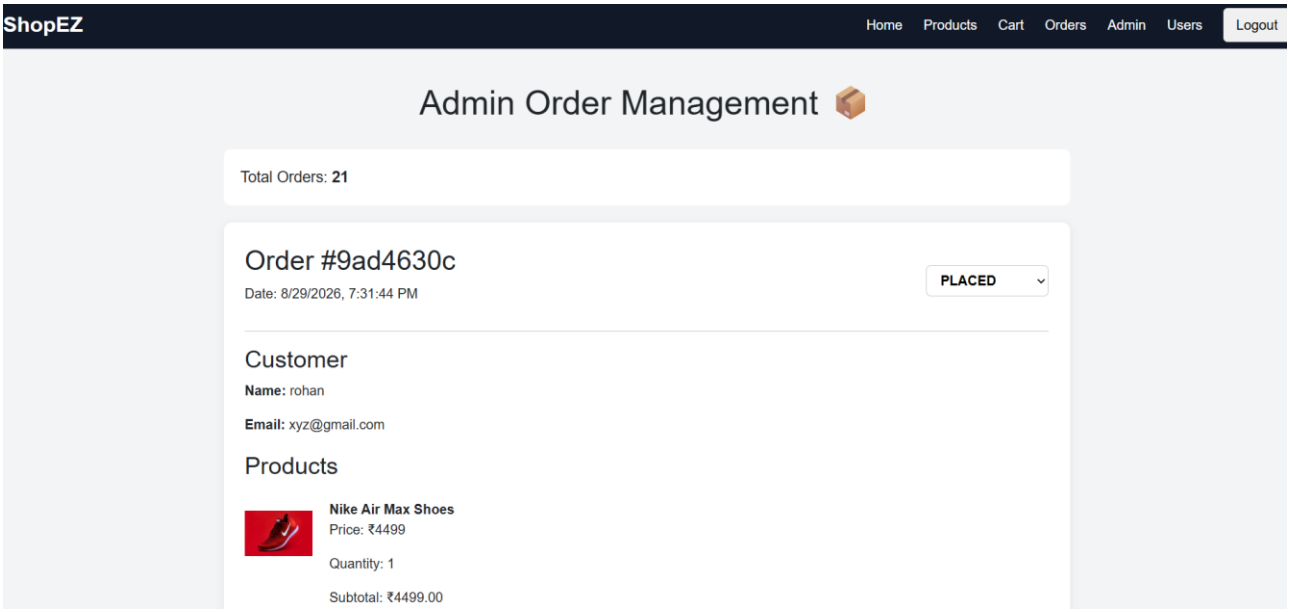
The administrator module is protected by authentication and role-based authorization. Admin routes use role middleware so ordinary users cannot access administrative operations.

Admin Function	Description
Dashboard	Displays users, products, orders, pending orders, delivered orders, cancelled orders and total sales.
Products	Create, update and manage product catalog information.
Users	View/manage registered users and their roles.
Orders	View all customer orders.
Order Status	Update an order through PLACED, CONFIRMED, SHIPPED, DELIVERED or CANCELLED.
Inventory	Stock is reduced when orders are created and restored when eligible orders are cancelled.






ShopEZ
Home
Products
Cart
Orders
Admin
Users
Logout
User Management
Name
Email
Phone
Role
Action
rohan
xyz@gmail.com
4545454545
USER
Delete
prasad
prasad2026test@gmail.com
Prasad@123
ADMIN
Delete
prasad
prasadtest@gmail.com
9876543210
USER
Delete
Test User
testuser@gmail.com
9876543211
USER
Delete
Prasad
prasad@gmail.com
9876543210
USER
Delete


ShopEZ
Home
Products
Cart
Orders
Admin
Users
Logout
Admin Order Management
Total Orders: 21
Order #9ad4630c
Date: 8/29/2026, 7:31:44 PM
PLACED
Customer
Name: rohan
Email: xyz@gmail.com
Products
Nike Air Max Shoes
Price: ₹4499
Quantity: 1
Subtotal: ₹4499.00

11.Shopping Cart & Order Flow



The cart controller supports five main operations: add product, get cart, update quantity, remove product and clear cart. Each operation is associated with the authenticated user through req.user._id.

Stock handling: When adding or updating a cart item, the backend checks product stock. During checkout, stock is checked again to prevent orders that exceed available quantity.

Order creation: The order controller copies cart items into the order, calculates the total amount, stores the shipping address and payment method, reduces product stock, and clears the cart.

Cancellation: Customer cancellation is permitted only for PLACED and CONFIRMED orders. When an order is cancelled, item quantities are added back to product stock.

12. Payment Integration

Razorpay is integrated for online payments. The frontend dynamically loads the Razorpay checkout script. The backend creates a Razorpay order using the server-side secret key and returns the public key and payment order details to the frontend.

After payment, the frontend sends the Razorpay order ID, payment ID and signature to the backend. The backend creates an HMAC-SHA256 signature using the Razorpay key secret and compares it with the received signature. Only after successful verification is the ShopEZ order created with paymentStatus set to PAID.

For COD, the regular order endpoint creates the order with paymentMethod set to COD and paymentStatus set to PENDING.

Important security rule: Razorpay secret keys must never be placed in React source code or committed to GitHub. They belong only in server environment configuration.

13. REST API Documentation

The main API structure implemented by ShopEZ is summarized below. Protected endpoints require the authentication token handled by the backend middleware.

Authentication

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate a user and return token

Products

Method	Endpoint	Purpose
GET	/api/products	Get product list
GET	/api/products/:id	Get product details
POST / PUT / DELETE	/api/products/...	Product management operations depending on configured routes

Cart

Method	Endpoint	Purpose
POST	/api/cart/add	Add product to cart
GET	/api/cart	Get current user cart
PUT	/api/cart/update	Update cart item quantity
DELETE	/api/cart/remove/:productId	Remove a product from cart
DELETE	/api/cart/clear	Clear the cart

Orders

Method	Endpoint	Purpose
POST	/api/orders	Create COD/general order
GET	/api/orders/my-orders	Get logged-in user orders
GET	/api/orders/:id	Get one order; ownership/admin checked
PUT	/api/orders/cancel/:id	Cancel an eligible order
GET	/api/orders/admin/stats	Admin statistics
GET	/api/orders/admin/all	Get all orders for admin
PUT	/api/orders/admin/status/:id	Update order status

Payments

Method	Endpoint	Purpose

POST	/api/payment/create-order	Create Razorpay payment order
POST	/api/payment/verify	Verify payment signature and place paid order

The exact prefix depends on how routes are mounted in server.js. The frontend examples use paths such as /orders, /payment/create-order and /payment/verify through the configured Axios base URL.

14. Authentication & Security

ShopEZ uses authentication middleware to identify the logged-in user and protect private resources. Role middleware is used for administrator-only routes.

- JWT token is used for authenticated requests.
- Protected frontend pages can redirect unauthenticated users to login.
- Backend middleware verifies authentication before user-specific data is accessed.
- Admin operations are protected with role-based authorization.
- Order details include an ownership check so a normal user cannot view another user’s order.
- Razorpay payment signature is verified on the server using HMAC-SHA256.
- Sensitive environment variables such as database credentials and Razorpay secret keys are stored outsidesource code.
- Product stock is checked on the server instead of trusting only frontend values.

15. Validation & Error Handling

Field / Operation	Validation
Phone	10-digit Indian mobile number beginning with 6-9
Pincode	Exactly 6 digits
Shipping	Name, phone, address, city, state and pincode required
Cart quantity	Must be at least 1 and must not exceed stock
Product	Product must exist before it can be ordered
Payment	Razorpay order ID, payment ID and signature required
Order status	Must match an allowed status
Authorization	User must own the order unless request is from ADMIN

Common HTTP responses include 200/201 for successful operations, 400 for invalid input, 403 for forbidden access, 404 for missing resources, and 500 for unexpected server errors.

16. Testing

Testing was performed by exercising the major user and administrator workflows from the frontend and checking corresponding backend responses.

- User registration and login.
- Protected route access with and without authentication.
- Product listing and product details.
- Add to cart, update quantity, remove item and clear cart.
- Stock-limit validation.
- COD checkout and order creation.
- Razorpay order creation and payment verification.
- My Orders and Order Details pages.

- Customer cancellation of PLACED/CONFIRMED orders.
- Admin dashboard statistics.
- Admin order status changes.
- Stock restoration after cancellation.
- Error handling for invalid or missing data.

17. Installation & Setup

Prerequisites

- Node.js and npm installed.
- MongoDB running locally or a MongoDB connection string available.
- Git installed for version control.
- Razorpay test/live account credentials if online payment testing is required.

Step 1 - Clone the project

```
git clone https://github.com/prasad872005/ShopEZ.git
```

Step 2 - Install frontend dependencies

```
cd ShopEZ\client
npm install
```

Step 3 - Install backend dependencies

```
cd ..\server
npm install
```

Step 4 - Configure server environment variables

Create server/.env and add the required database, JWT and Razorpay configuration. Never commit this file to GitHub.

Step 5 - Start the backend

```
cd server
npm start
```

Step 6 - Start the frontend

```
cd client
npm run dev
```

Step 7 - Open the frontend URL shown by Vite in the terminal and test the complete workflow.

18. Environment Variables

Variable	Purpose	Example / Note
MONGO_URI	MongoDB connection string	Keep private; local or cloud URI
JWT_SECRET	JWT signing secret	Use a strong random value
RAZORPAY_KEY_ID	Razorpay public key	Can be returned to frontend when needed
RAZORPAY_KEY_SECRET	Razorpay server secret	Never expose in frontend or GitHub
PORT	Backend server port	For example 5000

The frontend may use an environment variable for its API base URL depending on the configured Axios service. Environment files should be excluded through .gitignore when they contain secrets.

19. Deployment & GitHub

The project source code is maintained in a GitHub repository. Git is used to track changes, create commits and push the project to the remote main branch.

Repository:

```
https://github.com/prasad872005/ShopEZ
```

Recommended repository contents include the client and server source code, package files, README documentation and project configuration. Node_modules and secret .env files should not be committed.

A professional README should contain the project overview, features, technology stack, setup instructions, API summary, screenshots/demo link and repository information.

20. Future Enhancements

- Add product search with advanced filters and sorting.
- Add wishlist functionality.
- Add coupon and discount-code management.
- Add email notifications for order placement and status updates.
- Add Razorpay webhook handling for stronger payment lifecycle tracking.
- Add automated refund handling for eligible online orders.
- Add pagination and improved product search performance.
- Add product image upload and cloud storage.
- Add analytics charts for sales and product performance.
- Add automated unit and integration test suites.
- Deploy frontend and backend to production hosting with managed MongoDB.
- Improve responsive design and accessibility across all pages.

21. Conclusion

ShopEZ successfully demonstrates the development of a complete MERN stack e-commerce application. It combines a React frontend with a Node.js/Express backend and MongoDB database, while also implementing authentication, role-based authorization, cart management, inventory validation, order processing, cancellation, administrative management and Razorpay payment verification.

The project provides practical experience with real-world full-stack concepts such as REST APIs, database relationships, middleware, protected routes, asynchronous operations, payment integration, validation and source-code management. It can therefore serve as an academic project, portfolio project and discussion point during software-development interviews.

22. References / Tools

- React.js - frontend component library/framework.
- Vite - frontend build and development tool.
- Node.js - JavaScript runtime.
- Express.js - backend web framework.
- MongoDB and Mongoose - database and ODM.
- Razorpay - online payment gateway.
- Axios - HTTP client used by the frontend.
- Git and GitHub - source control and repository hosting.

Project Documentation End

ShopEZ - MERN Stack E-Commerce Application